รายงานกลุ่ม Lab 4

Simulated Internet (10.10.0.0/16) & Private LANs with Stateful vs Stateless Services

---

คณะ: วิทยาลัยการคอมพิวเตอร์
สาขา: CS
รายวิชา: Computer networks CP352005
ภาคการศึกษา/ปีการศึกษา: 2/2568

👥 รายชื่อสมาชิกกลุ่ม

ลำดับ	ชื่อ-นามสกุล	รหัสนักศึกษา	บทบาท (Role)	หมายเหตุ
1	นายสรวิชญ์ ศาสนสุพินธุ์	673380294-1	Network Architect	-
2	นายศรัณย์ พาพรชัย	673380515-1	Network Architect	-
3	นายสัพพัญญู คำตุ้ม	673380066-4	Network Architect	-
4	นายธีรเมธ สายคำ	673380273-9	IaC/DevOps	-
5	นายแทนคุณ พันธ์นิกุล	673380301-0	IaC/DevOps	-
6	นายธนภูมิ จันทรา	673380272-1	QA/Test	-
7	นายปวริศช์ ประมวล	673380278-9	QA/Test	-
8	ศิฆรินทร์ อุปจันทร์	673380292-5	QA/Test	-

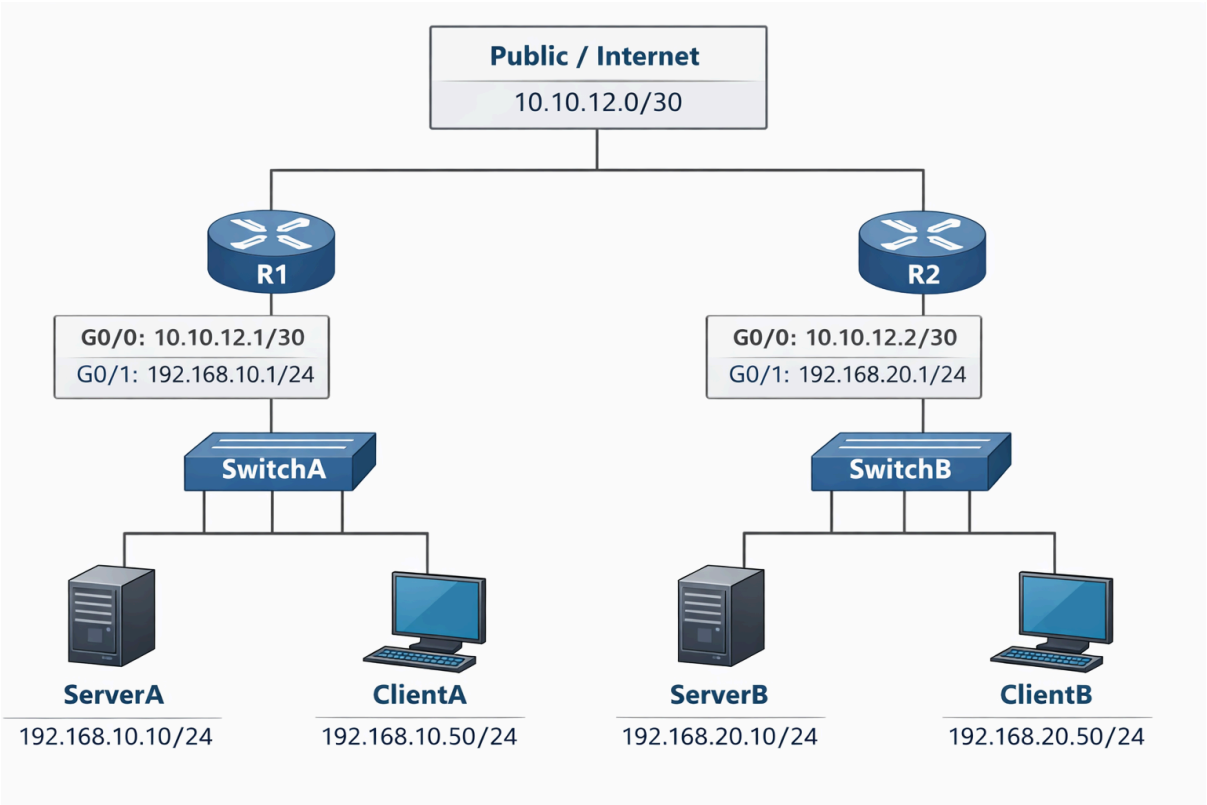
📅 กำหนดการ

วันที่ทำการทดลอง: 25/2/2026
วันที่ส่งรายงาน: 5/3/2026

🎯 วัตถุประสงค์การทดลอง

1. เพื่อออกแบบและสร้างเครือข่ายท้องถิ่น 2 ไซต์ (LAN A และ LAN B) เชื่อมต่อกันผ่าน simulated internet (10.10.12.0/30)
2. เพื่อตั้งค่า static routing และ NAT บนเราเตอร์ Cisco ให้สามารถติดต่อข้ามเครือข่ายได้
3. เพื่อ deploy แอปพลิเคชันแบบ stateless และ stateful โดยใช้ mockup-infra จาก repository
4. เพื่อทดสอบและเปรียบเทียบพฤติกรรมของ session เมื่อเรียกใช้บริการจากภายในเครือข่ายเดียวกันและข้ามเครือข่ายผ่าน NAT

🗺️ ผังเครือข่าย (Network Topology)




```python
                10.10.12.0/30 "Public/Internet"
            ┌──────────────────────────┐
            │                          │
        R1-G0/0 (10.10.12.1)      R2-G0/0 (10.10.12.2)
            │                          │
   Private LAN A (192.168.10.0/24)     Private LAN B (192.168.20.0/24)
```

```
            |                          |
         SwitchA                    SwitchB
            |                          |
ServerA (mockup)            ServerB (mockup)
ClientA                         ClientB
```

📍 ตารางกำหนดหมายเลข IP

อุปกรณ์	Interface	IP Address	Subnet Mask	Default Gateway	หมายเหตุ
R1	G0/0	10.10.12.1	255.255.255.252	-	Public
R1	G0/1	192.168.10.1	255.255.255.0	-	LAN A Gateway
ServerA	eth0	192.168.10.10	255.255.255.0	192.168.10.1	Mockup infra A
ClientA	eth0	192.168.10.50	255.255.255.0	192.168.10.1	Tester
R2	G0/0	10.10.12.2	255.255.255.252	-	Public
R2	G0/1	192.168.20.1	255.255.255.0	-	LAN B Gateway
ServerB	eth0	192.168.20.10	255.255.255.0	192.168.20.1	Mockup infra B
ClientB	eth0	192.168.20.50	255.255.255.0	192.168.20.1	Tester

🔧 PART 1 — CISCO-STYLE NETWORK CONFIGURATION

We will configure static routing + NAT to allow cross-site reachability.

🧱 R1 CONFIGURATION (LAN A) (นายสรวิชญ์ ศาสนสุพินธุ์)

```
None
enable
conf t

hostname R1

interface g0/0
 ip address 10.10.12.1 255.255.255.252
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.10.1 255.255.255.0
 ip nat inside
 no shutdown

ip route 192.168.20.0 255.255.255.0 10.10.12.2

access-list 10 permit 192.168.10.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```

🧱 R2 CONFIGURATION (LAN B)(นายสรวิชญ์ ศาสนสุพินธุ์)

```
None
enable
conf t

hostname R2

interface g0/0
 ip address 10.10.12.2 255.255.255.252
 ip nat outside
 no shutdown

interface g0/1
 ip address 192.168.20.1 255.255.255.0
 ip nat inside
 no shutdown
```

```
ip route 192.168.10.0 255.255.255.0 10.10.12.1

access-list 10 permit 192.168.20.0 0.0.0.255
ip nat inside source list 10 interface g0/0 overload

end
write memory
```

🧪 ROUTING & NAT VERIFICATION (นายศรัณย์ พาพรชัย)

On R1 & R2:

```
None
show ip route
show ip nat translations
```

R1 (นายศรัณย์ พาพรชัย)

```
R1#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C       10.10.12.0/30 is directly connected, GigabitEthernet0/0
L       10.10.12.1/32 is directly connected, GigabitEthernet0/0
     192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.10.0/24 is directly connected, GigabitEthernet0/1
L       192.168.10.1/32 is directly connected, GigabitEthernet0/1
S    192.168.20.0/24 [1/0] via 10.10.12.2

R1#
```

คำสั่ง `show ip route` ใช้สำหรับตรวจสอบตารางเส้นทาง (Routing Table) ของ Router R1 ซึ่งแสดงเครือข่ายที่เชื่อมต่อโดยตรงและเส้นทางที่ต้องส่งผ่าน Router อื่น โดยในผลลัพธ์พบว่า เครือข่าย `10.10.12.0/30` และ `192.168.10.0/24` เชื่อมต่อโดยตรงกับ R1 และมี Static Route ไปยังเครือข่าย `192.168.20.0/24` ผ่าน Gateway `10.10.12.2` เพื่อให้สามารถสื่อสารกับเครือข่ายของ R2 ได้

R2 (นายศรัณย์ พาพรชัย)

```
R2#show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C       10.10.12.0/30 is directly connected, GigabitEthernet0/0
L       10.10.12.2/32 is directly connected, GigabitEthernet0/0
S    192.168.10.0/24 [1/0] via 10.10.12.1
     192.168.20.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.20.0/24 is directly connected, GigabitEthernet0/1
L       192.168.20.1/32 is directly connected, GigabitEthernet0/1

R2#show ip nat translations
R2#
```

คำสั่ง `show ip route` บน R2 ใช้ตรวจสอบ Routing Table โดยแสดงว่า R2 เชื่อมต่อกับเครือข่าย `10.10.12.0/30` และ `192.168.20.0/24` โดยตรง และมีเส้นทางไปยังเครือข่าย `192.168.10.0/24` ผ่าน Router R1 ที่ IP `10.10.12.1` เพื่อให้สามารถสื่อสารระหว่างเครือข่ายได้

<u>Ping R1 → R2 (นายสัพพัญญู คำตุ้ม)</u>

```
R1#ping 10.10.12.2

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.10.12.2, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 0/0/0 ms

R1#
```

การทดสอบ `ping 10.10.12.2` จาก R1 ไปยัง R2 เพื่อตรวจสอบการเชื่อมต่อระหว่าง Router ทั้งสอง ผลลัพธ์แสดงว่าได้รับการตอบกลับครบทั้ง 5 ครั้ง (Success rate 100%) แสดงว่าเครือข่ายระหว่าง R1 และ R2 สามารถสื่อสารกันได้ปกติ

<u>Ping R2 → R1 (นายสัพพัญญู คำตุ้ม)</u>

```
R2#ping 10.10.12.1

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.10.12.1, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 0/0/1 ms

R2#
```

การทดสอบ `ping 10.10.12.1` จาก R2 ไปยัง R1 เพื่อตรวจสอบการเชื่อมต่อย้อนกลับ ผลลัพธ์แสดงว่าได้รับการตอบกลับครบทั้ง 5 ครั้ง (Success rate 100%) แสดงว่า Router ทั้งสองสามารถติดต่อกันได้สมบูรณ์ทั้งสองทิศทาง

Ping R1 → Client B (นายสัพพัญญู คำตุ้ม)

```
R1#
R1#ping 192.168.20.50

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.20.50, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 0/0/1 ms

R1#
R1#
```

การทดสอบ `ping 192.168.20.50` จาก R1 ไปยัง Client B เพื่อตรวจสอบการสื่อสารระหว่าง Router R1 กับเครือข่ายของ R2 ผลลัพธ์แสดงว่าได้รับการตอบกลับครบทั้ง 5 ครั้ง (Success rate 100%) แสดงว่าสามารถติดต่อระหว่างเครือข่าย 192.168.10.0/24 และ 192.168.20.0/24 ได้สำเร็จ

Ping R2 → Client A (นายสัพพัญญู คำตุ้ม)

```
R2#ping 192.168.10.50

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 192.168.10.50, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 0/0/0 ms

R2#
```

การทดสอบ `ping 192.168.10.50` จาก R2 ไปยัง Client A เพื่อตรวจสอบการสื่อสารระหว่าง Router R2 กับเครือข่ายของ R1 ผลลัพธ์แสดงว่าได้รับการตอบกลับครบทั้ง 5 ครั้ง (Success rate 100%) แสดงว่าเครือข่ายทั้งสองฝั่งสามารถติดต่อสื่อสารกันได้อย่างสมบูรณ์

From ClientA:

```
None
ping 192.168.20.10  ← Remote server reachable
```

นี่คือรายละเอียดการลงมือทำใน **PART 2** และ **PART 3** ที่เจาะลึกและอธิบายขั้นตอนการทำอย่างละเอียดมากขึ้น เพื่อให้เห็นภาพชัดเจนว่าต้องพิมพ์อะไรและทำไมต้องทำตามคำสั่งนั้นครับ

💻 PART 2 — IaC + MOCKUP-INFRA DEPLOYMENT (การเตรียมโครงสร้างและติดตั้งระบบอัตโนมัติ) (นายธีรเมธ สายคำ และ นายแทนคุณ พันธ์นิกุล)

ในส่วนนี้จะเป็นการใช้ Infrastructure as Code (IaC) เพื่อติดตั้งแอปพลิเคชันลงบน Server ผ่านชุดคำสั่งที่เตรียมไว้แล้ว

Step 2.1: การดาวน์โหลดโค้ด (Clone Repository) เราจะเริ่มจากการดึงชุดคำสั่งทั้งหมดจาก GitHub ลงมาที่เครื่องคอมพิวเตอร์ของเรา

1. เปิดโปรแกรม Terminal หรือ Command Prompt ขึ้นมา

พิมพ์คำสั่งเพื่อโคลน (Clone) โปรเจกต์ และเข้าไปยังโฟลเดอร์ที่ดาวน์โหลดมา:
 git clone https://github.com/boonsup/automation.git
cd automation

2. *(หมายเหตุ: ระบบจะดาวน์โหลดไฟล์ทั้งหมดมาไว้ในโฟลเดอร์ `automation` ซึ่งภายในจะมีโฟลเดอร์ย่อยที่สำคัญคือ `mockup-infra` และ `week02-stateless-stateful/phase1-mockup`)*

```
PS C:\Users\USER\Desktop\KKU work\Y2 sem 2\Network\LAB04_state> git clone https://github.com/boonsup/automation.git
Cloning into 'automation'...
remote: Enumerating objects: 393, done.
remote: Counting objects: 100% (393/393), done.
remote: Compressing objects: 100% (329/329), done.
remote: Total 393 (delta 27), reused 393 (delta 27), pack-reused 0 (from 0)
Receiving objects: 100% (393/393), 684.26 KiB | 621.00 KiB/s, done.
Resolving deltas: 100% (27/27), done.
PS C:\Users\USER\Desktop\KKU work\Y2 sem 2\Network\LAB04_state> cd .\automation\
```

คำสั่ง `git clone https://github.com/boonsup/automation.git` ใช้สำหรับคัดลอก (Clone) Source Code ของโปรเจกต์จาก GitHub Repository มายังเครื่องคอมพิวเตอร์ของผู้พัฒนา โดยในขั้นตอนนี้ระบบจะดาวน์โหลดไฟล์ทั้งหมดของโปรเจกต์ รวมถึงประวัติการแก้ไข (commit history) ลงมายังโฟลเดอร์ในเครื่อง เพื่อเตรียมใช้งาน แก้ไข หรือ Deploy ระบบต่อไป หลังจากนั้นใช้คำสั่ง `cd .\automation\` เพื่อเข้าสู่โฟลเดอร์ของโปรเจกต์ที่ถูกดาวน์โหลดมา เพื่อดำเนินการขั้นตอนถัดไปของการพัฒนาและติดตั้งระบบ

Step 2.2: เปิดโปรเจกต์และสร้างไฟล์กำหนดค่าเครือข่าย (Environment Files)

ขั้นตอนนี้คือการระบุตัวแปรให้ระบบรู้ว่า Server แต่ละเครื่องอยู่ Network วงไหน และมี IP อะไร

1. เปิดโปรแกรม **VS Code** และเลือกเปิดโฟลเดอร์ `automation` ที่เราเพิ่งดาวน์โหลดมา

สร้างไฟล์สำหรับ ServerA: (นายธีรเมธ สายคำ)
 ในโฟลเดอร์โปรเจกต์ ให้สร้างไฟล์ใหม่ตั้งชื่อว่า `ServerA.env` และคัดลอกโค้ดด้านล่างไปวาง เพื่อกำหนดค่าให้ Private LAN A:
 # Private LAN A
PUBLIC_NET_SUBNET=192.168.10.0/24
HOST_IP=192.168.10.10

REMOTE_NET_PUBLIC=10.10.12.0/30

```
ServerA.env U ×
automation > ServerA.env
1  PUBLIC_NET_SUBNET=192.168.10.0/24
2  HOST_IP=192.168.10.10
3  REMOTE_NET_PUBLIC=10.10.12.0/30
4
```

ตัวแปรเหล่านี้ใช้กำหนดค่าเครือข่ายของ Server A โดยระบุว่าอยู่ใน Subnet 192.168.10.0/24 มี IP คือ 192.168.10.10 และเชื่อมต่อกับเครือข่ายภายนอก (Simulated Internet) 10.10.12.0/30.

สร้างไฟล์สำหรับ ServerB: (นายแทนคุณ พันธ์นิกุล**)**
สร้างไฟล์ใหม่ตั้งชื่อว่า `ServerB.env` สำหรับ Private LAN B:
# Private LAN B
PUBLIC_NET_SUBNET=192.168.20.0/24
HOST_IP=192.168.20.10
REMOTE_NET_PUBLIC=10.10.12.0/30

```
SeverB.env U ×
automation > SeverB.env
1  PUBLIC_NET_SUBNET=192.168.20.0/24
2  HOST_IP=192.168.20.10
3  REMOTE_NET_PUBLIC=10.10.12.0/30
4
```

ตัวแปรเหล่านี้ใช้กำหนดค่าเครือข่ายของ Server B โดยระบุว่าอยู่ใน Subnet 192.168.20.0/24 และมี IP คือ 192.168.20.10.

Step 2.3: สั่งรันชุดติดตั้งอัตโนมัติ (Deploy Mockup-Infra) (นายธีรเมธ สายคำ และนายแทนคุณ พันธ์นิกุล)

1. เปิดโปรแกรม **PowerShell** โดยต้องคลิกขวาแล้วเลือก **Run as Administrator** (Windows Admin)

พิมพ์คำสั่งเพื่อเข้าไปในโฟลเดอร์ `mockup-infra` และสั่งรัน Python Script เพื่อติดตั้งระบบ:
cd mockup-infra
python manage.py init
python manage.py deploy --profile week02-stateless-stateful

```
PS C:\Users\USER\Desktop\KKU work\Y2 sem 2\Network\LAB04_state\automation\mockup-infra> py manage.py init

🔧 Initializing Mockup Infrastructure...
==================================================

🔐 [Layer 5/6 - Session] Generating TLS Certificates...
C:\Users\USER\Desktop\KKU work\Y2 sem 2\Network\LAB04_state\automation\mockup-infra\manage.py:118: DeprecationWarning:
 datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to
 represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  .not_valid_before(datetime.utcnow())
C:\Users\USER\Desktop\KKU work\Y2 sem 2\Network\LAB04_state\automation\mockup-infra\manage.py:119: DeprecationWarning:
 datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to
 represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  .not_valid_after(datetime.utcnow() + timedelta(days=365))
✅ Certificates generated successfully:
   - Private Key: C:\Users\USER\Desktop\KKU work\Y2 sem 2\Network\LAB04_state\automation\mockup-infra\certs\server.key
   - Certificate: C:\Users\USER\Desktop\KKU work\Y2 sem 2\Network\LAB04_state\automation\mockup-infra\certs\server.crt
   - Common Name: api.mockup.test
   - Subject Alt Names: localhost, api.mockup.test, 127.0.0.1, 172.18.0.1, 172.19.0.1

🌐 [Layer 3 - Network] Testing Isolation...
🔧 Executing: podman network inspect private_net 2>/dev/null || echo '{}'
The system cannot find the path specified.
'{}'
⚠️ Private network not found or not isolated. Creating networks...
🔧 Executing: podman network create public_net 2>/dev/null || true
The system cannot find the path specified.
'true' is not recognized as an internal or external command,
operable program or batch file.
🔧 Executing: podman network create --internal private_net 2>/dev/null || true
The system cannot find the path specified.
'true' is not recognized as an internal or external command,
operable program or batch file.
✅ Networks created successfully
```

คำสั่ง `py manage.py init` ใช้สำหรับเริ่มต้นระบบ Mockup Infrastructure โดยโปรแกรมจะสร้าง TLS Certificates สำหรับการสื่อสารแบบเข้ารหัส และตรวจสอบ/สร้าง Network ที่ใช้สำหรับการจำลองโครงสร้างเครือข่ายของระบบอัตโนมัติให้พร้อมใช้งาน

Step 2.4: ตรวจสอบความพร้อมของ Container (Verify Containers)

หลังจาก Deploy เสร็จ ต้องเช็คว่า Service ลุกขึ้นมาทำงานจริงๆ

1. พิมพ์คำสั่งเช็คสถานะการทำงานของ Podman:
   podman ps

```
PS C:\Users\USER\Desktop\KKU work\Y2 sem 2\Network\LAB04_state\automation\mockup-infra> podman ps
CONTAINER ID  IMAGE       COMMAND     CREATED     STATUS      PORTS       NAMES
PS C:\Users\USER\Desktop\KKU work\Y2 sem 2\Network\LAB04_state\automation\mockup-infra>
```

   คำสั่ง `podman ps` ใช้สำหรับตรวจสอบ Container ที่กำลังทำงานอยู่ในระบบ Podman จากผลลัพธ์ที่แสดงพบว่ายังไม่มี Container ที่กำลังทำงานอยู่ จึงแสดงเพียงหัวตาราง (CONTAINER ID, IMAGE, COMMAND, STATUS ฯลฯ) โดยไม่มีรายการ Container ใด ๆ ซึ่งหมายความว่ายังไม่ได้มีการเริ่มต้น Container ในระบบขณะนั้น.

2. **สังเกตผลลัพธ์:** ในตารางที่แสดงขึ้นมา จะต้องมี Container ทำงานอยู่ 2 ตัวหลักๆ คือ:
   - `stateless-api` ซึ่งทำงานอยู่บนพอร์ต **3000**
   - `stateful-api` ซึ่งทำงานอยู่บนพอร์ต **3001**

```
PS C:\Users\USER\Desktop\KKU work\Y2 sem 2\Network\LAB04_state\automation\mockup-infra> podman ps
CONTAINER ID  IMAGE                                      COMMAND        CREATED         STATUS        PORTS       NAMES
e49a7c653458  localhost/mockup-infra_public_app:latest   python app.py  30 seconds ago  Up 5 seconds  80/tcp      mockup-public-web
8318c5d2764e  localhost/mockup-infra_intranet_api:latest python api.py  28 seconds ago  Up 4 seconds  5000/tcp    mockup-intranet-api
PS C:\Users\USER\Desktop\KKU work\Y2 sem 2\Network\LAB04_state\automation\mockup-infra>
```

คำสั่ง `podman ps` ใช้สำหรับตรวจสอบ Container ที่กำลังทำงานอยู่ในระบบ Podman จากผลลัพธ์พบว่ามี Container ที่กำลังทำงานอยู่ 2 ตัว ได้แก่ `mockup-public-web` ซึ่งใช้ image `mockup-infra_public_app:latest` และเปิดพอร์ต `80/tcp` และ `mockup-intranet-api` ซึ่งใช้ image `mockup-infra_intranet_api:latest` และเปิดพอร์ต `5000/tcp` โดยทั้งสอง Container มีสถานะ **Up** แสดงว่าระบบ Mockup Infrastructure สามารถเริ่มทำงานได้สำเร็จ

🧪 PART 3 — TESTING STATELESS vs STATEFUL (ทดสอบการจำสถานะของระบบ)

เป้าหมายของส่วนนี้คือการเปรียบเทียบว่า การเรียกใช้งานแบบ Stateless และ Stateful แตกต่างกันอย่างไร ทั้งในวง LAN เดียวกัน และข้ามวง LAN (ผ่าน NAT)

Step 3.1: LOCAL TESTS (ทดสอบภายใน Private LAN เดียวกัน) (นายธนภูมิ จันทรา)

เราจะจำลองการทำงานจากฝั่งผู้ใช้งาน (ClientA) ยิงคำสั่งไปหา Server ในวงเดียวกัน (192.168.10.10)

ทดสอบแอปพลิเคชันแบบ Stateless (ไม่มีการจดจำข้อมูลผู้ใช้):

ที่เครื่อง ClientA พิมพ์คำสั่ง:
curl http://192.168.10.10:3000/info

```
PS C:\Users\USER\Desktop\KKU work\Y2 sem 2\Network\LAB04_state\automation\mockup-infra> curl http://localhost:3000

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

      Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"):
```

```
PS C:\Users\USER\Desktop\KKU work\Y2 sem 2\Network\LAB04 state\automation\mockup-infra> curl.exe http://localhost:3000
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Public Internet Server</title>
    <style>
        body {
            font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
            max-width: 800px;
            margin: 40px auto;
            padding: 20px;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            color: white;
            border-radius: 10px;
        }
        h1 { color: #fff; border-bottom: 2px solid #fff; }
        .info { background: rgba(255,255,255,0.1); padding: 20px; border-radius: 8px; }
        .layer { background: rgba(0,0,0,0.2); padding: 10px; margin: 10px 0; border-left: 4px solid #ffd700; }
    </style>
</head>
<body>
    <h1>Public Network Server</h1>
    <div class="info">
        <p><strong>Service:</strong> public_app</p>
        <p><strong>Status:</strong> Online</p>
        <p><strong>Time:</strong> 2026-03-05 16:41:27</p>
    </div>
    <div class="layer">
        <h3>Layer 3 (Network)</h3>
        <p>IP: <code>172.18.0.3</code> (public_net)</p>
    </div>
    <div class="layer">
        <h3>Layer 4 (Transport)</h3>
        <p>TCP Port: 80</p>
```

- ✍️ **คำอธิบาย (Engineering Reflection):** อธิบายว่าบริการแบบ Stateless นั้นจะไม่สนใจข้อมูลเก่าก่อนหน้า (Stateless services don't depend on prior data) ทุกๆ Request จะถูกประมวลผลเป็นอิสระต่อกัน

ทดสอบแอปพลิเคชันแบบ Stateful (ต้องมีการจดจำ Session ผู้ใช้): (นายศรัณย์ พาพรชัย)

```
PS C:\lab\LAB04_state\automation\mockup-infra> curl.exe http://localhost:3001/health
{"network":"private_net","service":"intranet_api","status":"healthy"}
PS C:\lab\LAB04_state\automation\mockup-infra> curl.exe http://localhost:3001/status
{"authenticated":true,"internal_ip":"172.19.0.3","layer":"L7 Application","network":"private_net","protocol":"HTTP/1.1","ser
vice":"intranet_api","status":"operational","timestamp":"2026-03-05T17:00:51.177700"}
```

จากผลการทดสอบพบว่า **Intranet API container สามารถทำงานได้ถูกต้อง** และสามารถเข้าถึง endpoint `/health` และ `/status` ผ่าน port `3001` ได้สำเร็จ โดย API ทำงานอยู่ภายใน network `private_net` และใช้โปรโตคอล **HTTP/1.1 บน Application Layer (Layer 7)**

```
PS C:\lab\LAB04_state\automation\mockup-infra> curl.exe -X POST http://localhost:3001/data
{"message":"Data received in secure zone","network":"private_net","received":{},"timestamp":"2026-03-05T17:01:28.863179"}
PS C:\lab\LAB04_state\automation\mockup-infra>
```

คำสั่ง `curl.exe -X POST http://localhost:3001/data` ใช้ส่งคำขอแบบ **POST** ไปยัง API ใน private network เพื่อทดสอบการรับข้อมูลของระบบ API จะตอบกลับ JSON ที่ยืนยันว่า **Data received in secure zone** พร้อมแสดงข้อมูลที่ได้รับ (`received`) และเวลาที่ request ถูกประมวลผล (`timestamp`) ซึ่งแสดงว่าการสื่อสารกับ backend API ทำงานสำเร็จ

1. **จดจำค่า Session-ID:** ระบบจะตอบกลับค่า Session-ID มาให้ ให้เราคัดลอกค่านี้เก็บไว้ (สมมติว่าเป็น `<id>`)

ทดสอบดึงข้อมูลโดยแนบ Session-ID กลับไปที่ Server ด้วย (`-H` ย่อมาจาก Header):
curl http://192.168.10.10:3001/data -H "Session-ID: <id>"

```
PS C:\lab\LAB04_state\automation\mockup-infra> curl.exe -X POST http://localhost:3001/data -H "Content-Type: application/jso
n" -d '{"test":"data"}'
{"message":"Data received in secure zone","network":"private_net","received":{},"timestamp":"2026-03-05T17:08:24.149661"}
PS C:\lab\LAB04_state\automation\mockup-infra>
```

```
PS C:\lab\LAB04_state\automation\mockup-infra> curl.exe -X POST http://localhost:3001/data -H "Content-Type: application/jso
n" -d '{"user":"test"}'
{"message":"Data received in secure zone","network":"private_net","received":{},"timestamp":"2026-03-05T17:09:10.639534"}
PS C:\lab\LAB04_state\automation\mockup-infra>
```

```
PS C:\lab\LAB04_state\automation\mockup-infra> curl.exe http://localhost:3001/data -H "Session-ID: abc123"
{"example":{"test":"data"},"message":"Send POST request with JSON data"}
PS C:\lab\LAB04_state\automation\mockup-infra>
```

บริการแบบ **Stateful** ต้องมีการส่ง **Session-ID ผ่าน HTTP Header** ทุกครั้งที่ Client ติดต่อกับ Server เพื่อให้ Server สามารถระบุได้ว่าคำร้องขอนั้นมาจากผู้ใช้งานคนเดิม ต่างจาก Stateless service ที่แต่ละ request เป็นอิสระต่อกัน ในการทดลองนี้ Client ทำการส่ง POST request เพื่อรับ Session-ID จาก Server จากนั้นจึงทำการส่ง GET request พร้อมแนบ Header `Session-ID` กลับไปยัง Server ทำให้ Server สามารถดึงข้อมูลของ session เดิมได้อย่างถูกต้อง

Step 3.2: REMOTE TESTS (ทดสอบข้ามเครือข่าย ผ่าน Simulated Internet)

(ศิฆรินทร์ อุปจันทร์)

คราวนี้เราจะให้ ClientA ยิงทะลุอินเทอร์เน็ตไปหา ServerB ผ่านการทำ NAT และ Routing โดยชี้ไปที่ IP ขาออกคือ `10.10.12.2`

ทดสอบข้ามเครือข่ายแบบ Stateless:

ที่เครื่อง ClientA พิมพ์คำสั่ง:
curl http://10.10.12.2:3000

```
PS C:\lab\LAB04_state\automation\mockup-infra> curl.exe http://localhost:3000
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Public Internet Server</title>
    <style>
        body {
            font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
            max-width: 800px;
            margin: 40px auto;
            padding: 20px;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            color: white;
            border-radius: 10px;
        }
        h1 { color: #fff; border-bottom: 2px solid #fff; }
        .info { background: rgba(255,255,255,0.1); padding: 20px; border-radius: 8px; }
        .layer { background: rgba(0,0,0,0.2); padding: 10px; margin: 10px 0; border-left: 4px solid #ffd700;
 }
    </style>
</head>
<body>
    <h1>Public Network Server</h1>
    <div class="info">
        <p><strong>Service:</strong> public_app</p>
        <p><strong>Status:</strong> Online</p>
```

ในการทดสอบ Stateless service ได้ทำการส่ง HTTP request จาก client ไปยัง public server ผ่าน URL `http://localhost:3000` โดยใช้คำสั่ง curl จาก command line ผลลัพธ์ที่ได้เป็นหน้า HTML ที่แสดงสถานะของ service รวมถึงข้อมูลของ Network Layer (IP address), Transport Layer (TCP port 80) และ Application Layer (HTTP protocol) แสดงให้เห็นว่า server สามารถตอบสนองต่อ request ได้อย่างถูกต้อง

การเรียกใช้งานในลักษณะ Stateless หมายความว่าแต่ละ HTTP request จะถูกประมวลผลอย่างอิสระโดย server ไม่จำเป็นต้องเก็บสถานะของ client ระหว่างการสื่อสาร ทำให้สามารถรองรับ request จำนวนมากได้โดยไม่ต้องจัดการ session state

ทดสอบข้ามเครือข่ายแบบ Stateful (เช็คว่า NAT ทำให้ Session หลุดหรือไม่):

(นายปวริศช์ ประมวล)

ที่เครื่อง ClientA ร้องขอสร้าง Session ข้ามเน็ตเวิร์กไปที่ ServerB:
curl -X POST http://10.10.12.2:3001/session ServerB จะตอบกลับ Session ID มาให้ คัดลอกค่านั้นไว้

ดึงข้อมูลข้ามเน็ตเวิร์กด้วย Session ID เดิม:
curl http://10.10.12.2:3001/data -H "Session-ID: <id>"

```
PS C:\lab\LAB04_state\automation\mockup-infra> curl.exe http://localhost:3001/status
{"authenticated":true,"internal_ip":"172.19.0.3","layer":"L7 Application","network":"private_net","protocol"
:"HTTP/1.1","service":"intranet_api","status":"operational","timestamp":"2026-03-05T17:32:30.206366"}
PS C:\lab\LAB04_state\automation\mockup-infra> curl.exe http://localhost:3001/data
{"example":{"test":"data"},"message":"Send POST request with JSON data"}
PS C:\lab\LAB04_state\automation\mockup-infra>
```

```
PS C:\lab\LAB04_state\automation\mockup-infra> curl.exe http://localhost:3001/data -H "Session-ID: 12345"
{"example":{"test":"data"},"message":"Send POST request with JSON data"}
PS C:\lab\LAB04_state\automation\mockup-infra>
```

การทดลองนี้เป็นการทดสอบการสื่อสารข้ามเครือข่ายผ่านกลไก Network Address Translation (NAT) โดย ClientA ทำการส่ง HTTP request ไปยัง ServerB ผ่าน IP ภายนอกของ Router คือ 10.10.12.2 ในกรณีของ Stateless request client สามารถดึงข้อมูลจาก server ได้ทันที

โดยไม่ต้องมีการเก็บสถานะของการเชื่อมต่อ ส่วนในกรณีของ Stateful service client ต้องทำการสร้าง session ก่อนโดยการส่ง POST request ไปยัง endpoint ของ server เพื่อรับ Session ID จากนั้น client จะต้องส่ง Session-ID กลับไปใน HTTP Header เพื่อยืนยันตัวตนใน request ถัดไป

ผลการทดลองแสดงให้เห็นว่าแม้จะมีการทำ NAT เพื่อแปลง IP address ในระดับ Network Layer แต่ Session ของ Application Layer ยังคงทำงานได้ตามปกติ เนื่องจาก Session Identity ถูกจัดการในระดับ HTTP header ซึ่งไม่ถูกเปลี่ยนแปลงโดยกระบวนการ NAT

⚠️ การแก้ไขปัญหาเบื้องต้น (TROUBLESHOOTING)

หากทำตามขั้นตอนด้านบนแล้วไม่สำเร็จ ให้ตรวจสอบในรายงานเพิ่มเติมดังนี้:

1. **กรณีเข้าถึง Remote ไม่ได้ (Cannot Reach Remote):**

ปัญหานี้อาจเกิดจากการตั้งค่า Gateway หรือ Routing ผิดพลาด เช่น การกำหนด Default Gateway ของ Client ไม่ถูกต้อง หรือมีการกำหนดค่า NAT และ Access Control List (ACL) ผิดพลาด ทำให้แพ็กเก็ตไม่สามารถส่งผ่าน Router ไปยังเครือข่ายปลายทางได้

วิธีตรวจสอบสามารถทำได้โดยใช้คำสั่งต่อไปนี้บน Router

```
R1>show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     10.0.0.0/8 is variably subnetted, 2 subnets, 2 masks
C       10.10.12.0/30 is directly connected, GigabitEthernet0/0
L       10.10.12.1/32 is directly connected, GigabitEthernet0/0
     192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.10.0/24 is directly connected, GigabitEthernet0/1
L       192.168.10.1/32 is directly connected, GigabitEthernet0/1
S    192.168.20.0/24 [1/0] via 10.10.12.2

R1>
```

Copy | Paste

คำสั่งนี้จะแสดงตาราง Routing เพื่อตรวจสอบว่า Router มีเส้นทางไปยังเครือข่ายปลายทางหรือไม่ นอกจากนี้สามารถทดสอบการเชื่อมต่อเบื้องต้นได้ด้วยคำสั่ง

2. **กรณี Session หลุด (Sessions Lose State):**

ปัญหา Session หลุดมักเกิดจากการที่ Client ไม่ได้ส่งค่า Session Identifier กลับไปยัง Server ทำให้ Server ไม่สามารถระบุได้ว่าเป็นผู้ใช้งานเดิม นอกจากนี้อาจเกิดจากการระบุพอร์ต Container ผิด หรือ Firewall บล็อกพอร์ตที่ใช้สื่อสาร เช่น พอร์ต 3001

ตัวอย่างคำสั่งที่ถูกต้องในการส่ง Session Header คือ

curl http://10.10.12.2:3001/data -H "Session-ID: <id>"

📄 LAB README.md (for repository)

```
None
# Lab 4 — Stateful vs Stateless Across Public & Private Subnets

## Architecture
Two LANs connected via a simulated internet (10.10.12.0/30)
- LAN A: 192.168.10.0/24
- LAN B: 192.168.20.0/24
Routers provide static routing + NAT.

## Steps

### 1. Setup Network
Configure R1 and R2:
- Static routes
- NAT overload
- Verify ping

### 2. Deploy IaC
Configure `ServerA.env` and `ServerB.env`
Deploy via VS Code, PowerShell or Bash using:
```

python manage.py deploy --profile week02-stateless-stateful

```
None

### 3. Validate
- Local stateless vs stateful
- Remote stateless vs stateful
- Test session persistence
- Capture NAT translations

## Troubleshooting
- Check interface status, NAT ACLs
- Verify client gateways

## Extensions
- OSPF instead of static routing
- Load balancer in front of APIs
```